

Guía completa de comandos Git Bash para Azure DevOps

Aquí tienes todos los comandos que necesitas para trabajar con un repositorio en Azure DevOps. Voy a simular una URL ficticia de tu empresa para los ejemplos.

URL simulada del repositorio:

text

`https://dev.azure.com/miempresa/CoreAcseIX/_git/RepositorioFormularios`

Configuración inicial (solo una vez)

Antes de empezar, configura tu identidad en Git. Esto se hace **una sola vez** por máquina.

bash

```
git config --global user.name "Tu Nombre"
```

```
git config --global user.email "tu.correo@miempresa.com"
```

Verifica que quedó bien:

bash

```
git config --global --list
```

1. Clonar el repositorio (primera vez)

Esto **descarga todo el repositorio** a tu máquina. Solo se hace la primera vez.

Paso 1: Abre Git Bash y navega a la carpeta donde quieres guardar el proyecto

bash

```
cd /c/Users/tu_usuario/Documents/Proyectos
```

Nota: En Git Bash, las rutas de Windows se escriben con `/c/` en lugar de `C:\`. La barra invertida `\` se cambia por `/`.

Paso 2: Clona el repositorio

```
bash
```

```
git clone https://dev.azure.com/miempresa/CoreAcseIX/_git/RepositorioFormularios
```

Se creará una carpeta llamada `RepositorioFormularios` con todo el código.

Paso 3: Entra a la carpeta del repositorio

```
bash
```

```
cd RepositorioFormularios
```

Paso 4: Verifica que estás en el branch correcto

```
bash
```

```
git branch
```

Muestra todos los branches locales y marca con `*` el actual. Para ver también los remotos:

```
bash
```

```
git branch -a
```

Paso 5: Cambiar a un branch específico (si trabajas en uno distinto a `main` o `master`)

```
bash
```

```
git checkout desarrollo
```

O si es un branch nuevo que solo está en el remoto:

```
bash
```

```
git checkout -b desarrollo origin/desarrollo
```

2. Actualizar tu copia local (traer cambios del servidor)

Antes de empezar a trabajar **cada día**, actualiza tu copia local con lo último que hay en el servidor. Esto evita conflictos.

Paso 1: Verifica si hay cambios nuevos sin descargarlos

```
bash
```

```
git fetch
```

Paso 2: Ver qué cambios hay en el remoto vs tu local

```
bash
```

```
git status
```

Paso 3: Descargar y aplicar los cambios

```
bash
```

```
git pull origin main
```

Cambia `main` por el nombre del branch en el que estés trabajando (ej. `git pull origin desarrollo`).

Alternativa: Si quieres traer cambios sin hacer merge automático

```
bash
```

```
git fetch origin
```

```
git log HEAD..origin/main --online      # Ver qué commits vienen
```

```
git merge origin/main                  # Aplicar los cambios
```

3. Editar archivos

Aquí no hay comandos especiales. Solo **abre los archivos con tu editor** (Notepad++, VS Code, Forms Builder, PL/SQL Developer, etc.) y modifícalos.

Ver qué archivos has modificado

bash

```
git status
```

Te mostrará:

- Modified:** archivos que ya existían y cambiaste.
- Untracked:** archivos nuevos que Git aún no conoce.
- Deleted:** archivos que borraste.

Ver los cambios línea por línea

bash

```
git diff
```

O para un archivo específico:

bash

```
git diff ruta/al/archivo.fmb
```



4. Preparar los cambios (staging)

Antes de hacer commit, debes indicarle a Git **qué cambios quieres guardar**.

Opción A: Agregar todos los cambios

bash

```
git add .
```

Opción B: Agregar archivos específicos

bash

```
git add ruta/al/archivo1.fmb
```

```
git add ruta/al/archivo2.sql
```

Opción C: Agregar todos los cambios de una carpeta

```
bash
```

```
git add forms/
```

Ver qué quedó en staging (listo para commit)

```
bash
```

```
git status
```

Los archivos en verde están listos para commit. Los rojos aún no.

Quitar un archivo del staging (si te equivocaste)

```
bash
```

```
git reset ruta/al/archivo.fmb
```



5. Hacer commit (guardar cambios en tu copia local)

Un commit es como "sacar una foto" del estado de tus cambios.

```
bash
```

```
git commit -m "Agregado formulario de bloqueo de puntos de venta"
```

Buenas prácticas para el mensaje del commit

Tipo	Cuándo usarlo	Ejemplo
feat:	Nueva funcionalidad	feat: agregar formulario bloqueo PV
fix:	Corrección de bug	fix: corregir filtro de agencia
refactor:	Reorganización sin cambiar funcionalidad	refactor: simplificar trigger POST-QUERY
docs:	Documentación	docs: actualizar README de instalación

Tipo	Cuándo usarlo	Ejemplo
chore:	Tareas varias	chore: actualizar .gitignore

Ejemplo con formato profesional

bash

```
git commit -m "feat: agregar bloqueo automático de puntos de venta por mora"
```

Si quieres ver el historial de commits

bash

```
git log --oneline -10
```

Muestra los últimos 10 commits.

↑ 6. Subir cambios al servidor (push)

Ahora subes tus commits a Azure DevOps.

Primer push en un branch nuevo

bash

```
git push -u origin nombre_del_branch
```

La opción `-u` establece el branch remoto como predeterminado, así después solo escribes `git push`.

Push normal (branches ya configurados)

bash

```
git push
```

Push a un branch específico

```
bash
```

```
git push origin desarrollo
```

Si te pide credenciales

Azure DevOps te pedirá usuario y contraseña (o un **Personal Access Token**). Si no tienes token:

1. Ve a https://dev.azure.com/miempresa/_usersSettings/tokens
 2. Crea un token con permisos de Code (Read & Write)
 3. Úsalo como contraseña cuando Git lo pida.
-

7. Resolver conflictos (merge)

Si alguien más subió cambios a los mismos archivos que tú, tendrás un **conflicto**. Aquí te explico cómo resolverlo.

Paso 1: Intenta hacer pull

```
bash
```

```
git pull origin main
```

Si hay conflicto, verás un mensaje como:

```
text
```

```
CONFLICT (content): Merge conflict in ruta/al/archivo.sql
Automatic merge failed; fix conflicts and then commit the result.
```

Paso 2: Ver qué archivos tienen conflicto

```
bash
```

```
git status
```

Los archivos con conflicto aparecerán como `both modified`.

Paso 3: Abre el archivo conflictivo

Dentro verás marcas como estas:

```
text

<<<<<< HEAD
-- Tu versión (local)
SELECT * FROM punto_venta WHERE estado = 'ACTIVO';
=====
-- Versión del servidor
SELECT * FROM punto_venta WHERE estado = 'INACTIVO';
>>>>>> origin/main
```

Paso 4: Decide qué versión conservar

- Conservar tu versión:** Borra todo lo que está entre `=====` y `>>>>>> origin/main`.
- Conservar la del servidor:** Borra todo lo que está entre `<<<<<< HEAD` y `=====`.
- Combinar ambas:** Escribe la versión final que quieras.

Borra también las líneas `<<<<<<`, `=====`, `>>>>>>`.

Paso 5: Marca el conflicto como resuelto

```
bash

git add ruta/al/archivo.sql
```

Paso 6: Si hay más archivos en conflicto, repite los pasos 3-5

Paso 7: Termina el merge con un commit

```
bash

git commit -m "merge: resolver conflictos con main"
```

Paso 8: Sube los cambios

```
bash
```

```
git push
```



8. Trabajar con branches (recomendado)

Nunca trabajes directamente en `main` o `master`. Crea tu propio branch para cada tarea.

Crear un branch nuevo y cambiarte a él

```
bash
```

```
git checkout -b feat/bloqueo-pv
```

Ver en qué branch estás

```
bash
```

```
git branch
```

Cambiar a otro branch

```
bash
```

```
git checkout main
```

Subir un branch nuevo por primera vez

```
bash
```

```
git push -u origin feat/bloqueo-pv
```

Fusionar tu branch con `main` (después de terminar)

```
bash
```

```
git checkout main
```

```
git pull origin main
git merge feat/bloqueo-pv
git push origin main
```

Eliminar un branch local (después de fusionarlo)

```
bash

git branch -d feat/bloqueo-pv
```

Eliminar un branch remoto

```
bash

git push origin --delete feat/bloqueo-pv
```

9. Comandos útiles del día a día

Comando	Para qué sirve
git status	Ver el estado actual
git log --oneline -10	Ver los últimos 10 commits
git diff	Ver cambios sin preparar
git diff --staged	Ver cambios preparados
git fetch	Descargar cambios sin aplicarlos
git pull	Descargar y aplicar cambios
git stash	Guardar cambios temporalmente
git stash pop	Recuperar cambios guardados
git checkout -- archivo	Descartar cambios locales de un archivo
git reset --hard HEAD	Descartar TODOS los cambios locales (¡peligroso!)

10. Flujo completo típico (resumen)

Aquí tienes el flujo que harás cada día en tu trabajo:

```
bash

# 1. Ir a la carpeta del repositorio
cd /c/Users/tu_usuario/Documents/Proyectos/RepositorioFormularios

# 2. Asegurarte de estar en el branch correcto
git checkout desarrollo

# 3. Traer lo último del servidor
git pull origin desarrollo

# 4. Editar archivos con tu editor (Notepad++, VS Code, Forms Builder, etc.)

# 5. Ver qué cambió
git status

# 6. Agregar los cambios
git add .

# 7. Hacer commit
git commit -m "feat: agregar opción bloqueo PV al menú"

# 8. Subir al servidor
git push origin desarrollo
```

11. Casos especiales

Caso 1: Olvidaste hacer pull antes de editar y hay conflictos

```
bash

git stash                # Guardar tus cambios temporales
git pull origin main     # Traer lo del servidor
git stash pop            # Recuperar tus cambios (puede haber conflictos)
```

```
# Resolver conflictos
git add .
git commit -m "merge: resolver conflictos"
git push
```

Caso 2: Necesitas descartar TODOS tus cambios locales

```
bash
git fetch origin
git reset --hard origin/main
```

⚠ **Cuidado:** Esto borra todo lo que no hayas subido.

Caso 3: Subiste algo mal y quieres revertirlo

```
bash
git revert HEAD          # Crea un commit que deshace el último
git push origin main
```

Caso 4: Hiciste commit pero aún no push y quieres deshacerlo

```
bash
git reset --soft HEAD~1  # Deshace el commit pero mantiene los cambios
```

Caso 5: Necesitas descargar solo un archivo del servidor

```
bash
git checkout origin/main -- ruta/al/archivo.fmb
```

12. Autenticación con Azure DevOps (Personal Access Token)

Azure DevOps ya no acepta contraseña normal para Git. Debes crear un **Personal Access Token (PAT)**.

Paso 1: Crear el token

1. Entra a `https://dev.azure.com/miempresa/_usersSettings/tokens`
2. Clic en **"New Token"**
3. Nombre: `git-bash-local`
4. Scopes: marca **Code (Read & Write)**
5. Copia el token generado (solo se muestra una vez)

Paso 2: Usarlo

Cuando Git te pida credenciales:

- **Usuario:** tu correo o usuario de Azure DevOps
- **Contraseña:** pega el token

Paso 3 (Opcional): Guardarlo permanentemente

```
bash
```

```
git config --global credential.helper manager
```

En Windows, esto usa el Administrador de Credenciales para recordarlo.

Alternativa: Guardar el token en la URL

⚠ **No recomendado** por seguridad, pero funciona:

```
bash
```

```
git clone https://TU_TOKEN@dev.azure.com/miempresa/CoreAcseIX/_git/RepositorioFormularios
```

13. Resumen de la sintaxis esencial

Acción	Comando
Clonar repositorio	<code>git clone URL</code>

Acción	Comando
Ver estado	<code>git status</code>
Traer cambios del servidor	<code>git pull origin branch</code>
Crear branch	<code>git checkout -b nombre_branch</code>
Cambiar de branch	<code>git checkout nombre_branch</code>
Agregar todos los cambios	<code>git add .</code>
Agregar un archivo	<code>git add archivo</code>
Hacer commit	<code>git commit -m "mensaje"</code>
Subir cambios	<code>git push origin branch</code>
Ver historial	<code>git log --oneline</code>
Ver diferencias	<code>git diff</code>
Guardar temporalmente	<code>git stash</code>
Recuperar temporal	<code>git stash pop</code>
Deshacer commit (mantiene cambios)	<code>git reset --soft HEAD~1</code>
Descartar todo	<code>git reset --hard origin/main</code>

14. Configurar tu repositorio local (archivos útiles)

Crear `.gitignore` para no subir basura

En la raíz del repositorio, crea un archivo llamado `.gitignore` con este contenido (adaptado para Oracle Forms):

```
text

# Archivos compilados de Oracle Forms
*.fmx
*.mmx
*.plx
```

```
*.rex
```

```
# Archivos temporales
```

```
*.tmp
```

```
*.log
```

```
*.bak
```

```
# Carpetas de IDE
```

```
.vscode/
```

```
.idea/
```

```
# Sistema operativo
```

```
Thumbs.db
```

```
.DS_Store
```

Esto evita que se suban archivos compilados que pueden regenerarse.

Ver el remote configurado

```
bash
```

```
git remote -v
```

Debe mostrar algo como:

```
text
```

```
origin https://dev.azure.com/miempresa/CoreAcseIX/_git/RepositorioFormularios (fetch)
```

```
origin https://dev.azure.com/miempresa/CoreAcseIX/_git/RepositorioFormularios (push)
```

Cambiar la URL del remote (si la empresa cambia de servidor)

```
bash
```

```
git remote set-url origin
```

```
https://dev.azure.com/miempresa/NuevoRepo/_git/RepositorioFormularios
```



15. Consejos prácticos para tu día a día

1. **Antes de empezar a trabajar:** siempre `git pull origin branch`.

2. **Antes de cada commit:** `git status` para ver qué vas a subir.
3. **Nunca uses** `git push --force` en branches compartidos (rompes el trabajo de otros).
4. **Haz commits pequeños y frecuentes** en lugar de uno gigante al final.
5. **Escribe buenos mensajes de commit** (qué hiciste, no cómo lo hiciste).
6. **Trabaja siempre en branches propios**, nunca en `main` directamente.
7. **Antes de mergear a main**, haz `git pull` para asegurarte de tener lo último.
8. **Si tienes dudas, `git status` es tu mejor amigo.**

Con esta guía tienes todo lo necesario para trabajar con el repositorio de tu empresa en Azure DevOps usando Git Bash. Si te sale un mensaje de error específico cuando empieces a usarlo, pégamelo y te ayudo a resolverlo.